

Submission Map — ClinicalBridge

Project: ClinicalBridge: Bridging the Clinical Context Gap

Course: COP-3442 Prompt Engineering

University: Bahçeşehir University

Department: Artificial Intelligence Engineering Department

Professor: Binnur Kurt

Group Members: - Kenan Eliyan — 2286181 - Rama Tamimi — 2285460 - Joud Wardeh — 2282493

Educational prototype. All data is fictional. Not a real clinical tool. No diagnoses. Clinician review is always required.

This document maps each capstone requirement to its location in the repository so it can be reviewed quickly.

1. Project Overview

ClinicalBridge is an **educational proof-of-concept** built for COP-3442 Prompt Engineering. It uses **simulated data only** — every patient, alert, lab, and symptom is fictional. It is a **multi-agent system** (Alert Triage, EHR Retrieval, Anamnesis, and Synthesis agents, coordinated by an orchestrator) that **connects three data sources — EHR, RPM, and anamnesis** — when a remote-monitoring alert fires. From those sources it **produces a structured Clinical Context Brief** in which every claim is traceable to its source, uncertainty and missing data are surfaced, and conflicts are flagged. It is **not a real clinical tool**: it does not diagnose, it issues no orders, and it is designed to support — never replace — clinician judgment.

2. Core Deliverables Map

Required Deliverable	File or Folder	What it Contains
Project Report	reports/final_project_report.md	Full written report: problem, architecture, dataset, prompt methodology, evaluation, reflection, limitations
Working Prototype	main.py , pipeline.py , notebook/clinicalbridge_prototype.ipynb	CLI entry point, shared core pipeline/orchestrator, and the runnable notebook
Simulated Dataset	data/ , scenarios/	10 fictional patients (EHR + RPM + anamnesis) and 5 self-contained scenarios with gold briefs
Prompt Engineering Portfolio	reports/prompt_engineering_portfolio.md , prompts/	v1/v2/v3 prompt iteration with before/after analysis; the actual system prompts + changelogs
Evaluation Report	reports/evaluation_report.md , evaluation/	Realistic results, adversarial table, limitations; the 7-metric scoring code
Demonstration	docs/07_demo_walkthrough.md , outputs/demo_scenario_1_transcript.md , outputs/demo_scenario_3_transcript.md , notebook/clinicalbridge_prototype.ipynb	Demo script + speaking notes, saved demo transcripts, and the annotated notebook
Safety and Ethics	README.md , reports/final_project_report.md , every Clinical Context Brief	Disclaimers, ethical considerations and limitations, and the safety disclaimer embedded in each brief
Architecture Diagrams	docs/09_architecture_diagrams.md	Mermaid diagrams: system architecture, data flow, provider design, prompt iteration, evaluation pipeline
Manual Claude Workflow	MANUAL_CLAUDE_WORKFLOW.md	How Claude is used manually as the LLM engine without an API key
Adversarial Testing	ADVERSARIAL_TESTS.md , adversarial/	Three safe-failure tests with expected behavior, plus the input fixtures
LangChain RAG (EHR retrieval backend)	langchain_pipeline/ , pipeline.py	Genuine LangChain RAG: EHR -> Document objects -> RecursiveCharacterTextSplitter -> local all-MiniLM-L6-v2 embeddings -> FAISS -> configurable top-k retriever (--langchain-rag-demo); integrated behind the EHR Retrieval Agent. Local only, no API keys
RAG / Memory / Tools (M6/M7 concepts)	rag_retrieval.py , session_memory.py , tools.py , memory/	Pure-Python TF-IDF EHR retrieval (--rag-demo) — the documented fallback

Required Deliverable	File or Folder	What it Contains
		backend; session audit memory; and agent tool wrappers
Brief alignment (few-shot, taxonomy, escalation, bias, params, refs)	reports/prompt_engineering_portfolio.md , reports/final_project_report.md , reports/evaluation_report.md , ADVERSARIAL_TESTS.md , pipeline.py	≥3 few-shot examples/agent; official urgency mapping (Critical/Urgent/Routine/Informational); critical-alert escalation; Bias Awareness; expanded References; Model Parameter Rationale

Note: EHR retrieval uses a **genuine LangChain RAG pipeline** (local HuggingFace embeddings + FAISS, no API key); it degrades to the local TF-IDF retriever if the LangChain stack is not installed. The four-agent orchestration still runs in **no-API stored-output mode** and needs no API key — LangChain replaces only the retrieval backend, behind an abstraction.

PDF versions (for final submission)

Print-ready **PDF** versions of the written deliverables are included alongside the Markdown:

Document	PDF
Final Project Report	reports/final_project_report.pdf
Prompt Engineering Portfolio	reports/prompt_engineering_portfolio.pdf
Evaluation Report	reports/evaluation_report.pdf
Submission Map	SUBMISSION_MAP.pdf

The Markdown files remain the source of truth; the PDFs (and matching `.html` files used to generate them) are provided for easy reading/printing. To regenerate a PDF, open the `.html` file in a browser and use **Print** → **Save as PDF**.

3. How to Run

```

pip install -r requirements.txt
python main.py                # full pipeline: scorecard + sample brief + adversarial
                              # safety tests
python main.py --demo 1       # clean step-by-step demo walkthrough for Scenario 1
python main.py --demo 3       # clean step-by-step demo walkthrough for Scenario 3
python main.py --rag-demo 1    # local TF-IDF EHR retrieval + session-memory record
                              # (fallback backend)
python main.py --langchain-rag-demo 1  # GENUINE LangChain RAG: Documents + FAISS + local
                              # embeddings

```

```
jupyter notebook notebook/clinicalbridge_prototype.ipynb # the annotated notebook (Restart & Run All)
```

The only hard dependency for the core pipeline is `jsonschema`; the notebook additionally uses `jupyter`. The genuine LangChain RAG backend adds `langchain*`, `faiss-cpu`, and `sentence-transformers` (all local — **no API key**), and degrades gracefully if they are absent.

4. Recommended Demo Path

Use these two contrasting scenarios:

- **Scenario 1 — Missed Medication.** Shows **cross-source reasoning**: the system connects the RPM blood-pressure alert, the EHR medication/refill data, and the patient-reported lapse in the anamnesis into one traceable picture.
- **Scenario 3 — Silent Deterioration.** Shows **trend detection and escalation beyond raw device severity**: no single reading is dramatic, but the system reasons over a multi-day weight/SpO2 trend and escalates urgency despite the device rating it only "medium."

Together they demonstrate both the "assemble a clear story" case and the "catch what no single alert reveals" case. Run `python main.py --demo 1` and `python main.py --demo 3` (add `--save-transcript` to regenerate the saved transcripts in `outputs/`).

5. Important Safety Statement

- **All data is fictional.** No real patients and no real PHI.
 - **Not for real clinical use.** ClinicalBridge is an educational prototype with no regulatory clearance.
 - **No diagnoses.** Output is framed strictly as *factors to consider*.
 - **No treatment orders.** The brief ends in clinician *considerations*, never orders or prescriptions.
 - **Clinician review is always required.** A human stays in the loop at all times.
 - **High evaluation scores reflect controlled, simulated data — not clinical validation.** They show the prompts can behave correctly and safely on clean cases, not that the system works on real patients.
-

6. Final Submission Checklist

Include the following in the submission zip:

- [] `README.md`
- [] `SUBMISSION_MAP.md`
- [] `MANUAL_CLAUDE_WORKFLOW.md`
- [] `ADVERSARIAL_TESTS.md`
- [] `main.py`

- [] pipeline.py
- [] langchain_pipeline/
- [] rag_retrieval.py
- [] session_memory.py
- [] tools.py
- [] memory/
- [] requirements.txt
- [] notebook/
- [] data/
- [] scenarios/
- [] schemas/
- [] prompts/
- [] agent_outputs/
- [] evaluation/
- [] adversarial/
- [] outputs/
- [] docs/
- [] reports/

PDF versions of the reports (included for submission):

- [] reports/final_project_report.pdf
- [] reports/prompt_engineering_portfolio.pdf
- [] reports/evaluation_report.pdf
- [] SUBMISSION_MAP.pdf

Tip: exclude generated cache folders (`__pycache__/` , `.ipynb_checkpoints/`) from the zip — they are not part of the submission.